Processing Workflow - Training and Inference - Run the IPython notebook

1. **Input RAW Underwater Image**
    - Read the original underwater image from the input directory.
2. **Calculate RGB Statistics**
    - Compute average Red (R), Green (G), and Blue (B) channel intensities.

```
from rgb import process_images_and_save_csv

# Paths
folder_path = "RAW"
csv_save_path = "Res/Images_seethru_rawRGB_HSV_ST_raw_all_values.csv"

process_images_and_save_csv(folder_path, csv_save_path)

########  To choose Module 1 ###############
## If Avg_R < ( Avg_G/5 | Avg_B/5):
#      # Low-lit -> IIluminateNet
# Else:
#      # High-lit-> SEF
```

3. **Check Illumination Condition**
    - Evaluate the following condition: Avg_R < Avg_G/5 or Avg_R < Avg_B/5
4. **If the Condition is TRUE (Low-Light Image)**
    - Generate Transmission Maps.

```
from transmap_generate import process_images

process_images(input_dir="RAW",
               output_dir="Transmap")
```

    - Use the RAW image and corresponding Transmission Map as inputs to **IlluminateNet (trainable network)**.
    - Obtain an illumination-enhanced image.

```
##  TRAINING
!python3 illuminatenet_main.py --images "RAW" \
                    --depth "Transmap" \
                    --output "IlluminateNet_output" \
                    --checkpoints "IlluminateNet_output/check"
```

```
##  INFERENCE
!python3 illuminatenet_inf.py --images "RAW" \
                    --depth "Transmap" \
                    --output 'IlluminateNet_inf' \
                    --checkpoints 'IlluminateNet_output/check'
```

5. **If the Condition is FALSE (Normal Illumination Image)**
    - Skip IlluminateNet.
    - Apply the **Spectral Equalization Filter (SEF)** directly to the RAW image.
    - Obtain a color-corrected image.

```python
import cv2 as cv
import os
from aocm_algorithm import spectral_equalization_filter

# --- Paths ---
input_folder = 'RAW'
output_folder = 'SEF_output'

os.makedirs(output_folder, exist_ok=True)

# --- Process all images in folder ---
valid_ext = (".jpg", ".jpeg", ".png", ".bmp", ".tif", ".tiff")

for filename in os.listdir(input_folder):
    if filename.lower().endswith(valid_ext):
        in_path = os.path.join(input_folder, filename)
        out_path = os.path.join(output_folder, filename)

        img = cv.imread(in_path)
        if img is None:
            print(f" Skipped (can't read): {filename}")
            continue

        neutralized = spectral_equalization_filter(img)
        cv.imwrite(out_path, neutralized)
        print(f"Saved: {out_path}")
```

6. **Apply AOCM Enhancement**
    - Use the output from either:
        i. IlluminateNet (low-light branch), or
        ii. SEF (normal-light branch).
    - Enhance image contrast, visibility, and color consistency using **AOCM**.

```python
from aocm_algorithm import AOCM
import cv2
import os

# input_folder = 'IlluminateNet_output'   # for low-lit
input_folder = 'SEF_output'
output_folder = 'AOCM_output'
os.makedirs(output_folder, exist_ok=True)

os.makedirs(output_folder, exist_ok=True)

for filename in os.listdir(input_folder):
    if filename.lower().endswith((".png", ".jpg", ".jpeg", ".bmp")):
        img_path = os.path.join(input_folder, filename)
        img = cv2.imread(img_path)
        if img is None:
            print(f"Skipping {filename}: cannot read")
            continue

        stem = os.path.splitext(filename)[0]

        # Process and save only the final AOCM output
        AOCM(img, return_all=False, save_path=output_folder, stem=stem)
        print(f"Processed {filename}")
```

7. **Generate Depth Maps**
    - Pass the AOCM-enhanced image to **Depth Anything**.
    - Generate a corresponding depth map.

```python
from depth_generation import depth_gen

depth_gen(input_folder = "AOCM_output/",output_folder = "depths")
```

8. **Run HydroOpticNet**
    - Use:
        i. AOCM-enhanced image
        ii. Generated depth map

      as inputs to **HydroOpticNet** (trainable network).

```
# TRAINING
!python3 hydroopticnet_main.py --images 'AOCM_output' \
                        --depth 'depths' \
                        --output "Hydroopticnet_output" \
                        --checkpoints 'Hydroopticnet_output/check'
```

```
# ## INFERENCE
!python3 hydroopticnet_inf.py --images 'AOCM_output' \
                        --depth 'depths' \
                        --output "Hydroopticnet_inf" \
                        --checkpoints 'Hydroopticnet_output/check'
```

9. **Generate Final Enhanced Output**
    - HydroOpticNet produces the final underwater image with improved illumination, color restoration, contrast, and visibility.

NOTE: For training: run cells with ## **TRAINING,** else comment it out if only inference is needed